Security Assessment Report

MEET-AI — AI Meeting Platform

Next.js / React / tRPC / Drizzle ORM / LiveKit / Stream Chat / Polar

Static source-code review — defensive analysis only. No application code was modified.

Scope of requested review

This assessment evaluates the MEET-AI codebase for the vulnerability classes listed below, plus any high-impact issues found along the way. Analysis is static (source review of branch main, commit 44abe85). No dynamic testing or exploitation was performed, and no code was changed.

- Server-Side Template Injection (SSTI)
- Regular-Expression Denial of Service (ReDoS)
- Layer-7 / low-and-slow Denial of Service (LpDoS)
- Secret-key leakage / sensitive-data exposure
- SQL / NoSQL injection
- Clipboard hijacking
- Replay attacks

Engagement details

Target: MEET-AI (theabdullahnadeem/MEET-AI)
Branch / commit: main @ 44abe85
Method: Manual static source review + dependency audit (npm audit)
Date: 2026-06-18

1. Executive Summary

MEET-AI is built on a modern, largely safe-by-default stack: database access goes through Drizzle ORM (parameterised queries), React/JSX auto-escapes output, and the tRPC data layer consistently scopes authorisation to the owning user. Consequently several requested classes — SQL/NoSQL injection, SSTI and clipboard hijacking — were not found to be exploitable in application code.

The review did, however, identify one high-impact access-control flaw and several hardening gaps. The most serious issue is that the LiveKit token endpoint mints a join token for ANY meeting room to ANY authenticated user, with no check that the caller belongs to that meeting. Combined with publicly-readable recording/ transcript storage, a registered user can join — or download the contents of — meetings that are not theirs.

Findings by severity

2 HIGH • **4 MEDIUM** • **3 LOW** • **1 INFO** • **3 NOT PRESENT**

Findings overview

F-01	HIGH	Broken access control / IDOR on LiveKit token endpoint
F-02	HIGH	Authentication secret not enforced (BETTER_AUTH_SECRET)
F-03	MEDIUM	Publicly-readable recording & transcript storage (R2)
F-04	MEDIUM	No rate limiting / request limits (LpDoS)
F-05	MEDIUM	Missing HTTP security headers (CSP / HSTS / framing)
F-06	MEDIUM	Vulnerable dependencies (npm audit: 1 critical, 33 high)
F-07	LOW	Webhook replay — no nonce / idempotency
F-08	LOW	Unescaped SQL LIKE wildcards in search
F-09	LOW	Polar billing client in sandbox mode
F-10	NOT PRESENT	SSTI & ReDoS (application code)
F-11	NOT PRESENT	SQL / NoSQL injection
F-12	NOT PRESENT	Clipboard hijacking
F-13	INFO	Prompt injection into LLM summary / chat (informational)

2. Methodology & Scope

The following surfaces were read and analysed line-by-line:

- Authentication & session: src/lib/auth.ts, src/trpc/init.ts (protectedProcedure / premiumProcedure).
- API routes: src/app/api/livekit-token, /livekit-webhook, /webhook.
- Data layer: src/modules/{meetings,agents}/server/procedures.ts, src/db/*.
- Background jobs & agent: src/inngest/function.ts, src/agents/meeting-agent.ts.
- Secret handling: src/lib/{livekit,stream-chat,stream-video,polar}.ts and all process.env references.
- Client rendering sinks: react-markdown, dangerouslySetInnerHTML, clipboard APIs, regular expressions.
- Configuration: next.config.ts, .env.example; plus a full dependency audit (npm audit).

Severities use a CVSS-style qualitative scale (HIGH / MEDIUM / LOW / INFO). "NOT PRESENT" means the class was specifically searched for and no exploitable instance was found in application code — not that it is impossible.

3. Detailed Findings

F-01 [HIGH] Broken access control / IDOR — LiveKit token endpoint

Status Present — exploitable by any authenticated user

Location *src/app/api/livekit-token/route.ts (GET, lines 7-30)*

What we found

The endpoint accepts a `room` query parameter and returns a LiveKit access token granting roomJoin, canPublish and canSubscribe for that room. It verifies only that a session exists — it never checks that the caller owns, created or was invited to the meeting whose id equals `room`. Room names ARE the meeting id (a nanoid shared with every participant and present in URLs). A registered user can therefore call /api/livekit-token?room=<anyMeetingId> and use the returned token with the public NEXT_PUBLIC_LIVEKIT_URL to connect to any meeting via the LiveKit client SDK, bypassing the owner-only meeting UI entirely.

Impact

Confidentiality and integrity of every meeting: an attacker can silently join, listen to and record other users' live meetings, publish disruptive audio/video, or screen-share. This is the highest-risk issue in the codebase.

Recommended fix

Before issuing the token, load the meeting by id and authorise the caller — e.g. confirm meetings.userId === session.user.id (current single-owner model), or that an approved membership row exists once the planned knock-to-join feature lands. Return 403 otherwise, and scope the grant (e.g. canPublish only for admitted participants).

F-02 [HIGH] Authentication secret not enforced in code

Status Conditional — critical if BETTER_AUTH_SECRET is unset in production

Location *src/lib/auth.ts (betterAuth config has no `secret` option)*

What we found

The better-auth instance sets no `secret` and relies entirely on the BETTER_AUTH_SECRET environment variable. If that variable is missing, better-auth falls back to a built-in default secret (the build logs emit "You are using the default secret"). Session tokens are signed with this key.

Impact

If the default/empty secret is used in production, anyone who knows the well-known default can forge valid session cookies and impersonate any user — a full authentication bypass.

Recommended fix

Set a strong, unique BETTER_AUTH_SECRET in production and fail fast at boot if it is absent (throw when process.env.BETTER_AUTH_SECRET is undefined, mirroring the guard already used in src/lib/livekit.ts). Add it to .env.example.

F-03 [MEDIUM] Publicly-readable recording & transcript storage

Status Present — by design (public r2.dev bucket)

Location *Cloudflare R2 public bucket; URLs built in src/agents/meeting-agent.ts and src/app/api/livekit-webhook/route.ts*

What we found

Recordings (recordings/<meetingId>.mp4) and transcripts (transcripts/<meetingId>.jsonl) are written to an R2 bucket exposed via a public r2.dev domain, at deterministic paths keyed by meeting id. Anyone who obtains a meeting id — every participant does, and it appears in client requests/logs — can fetch the full recording and transcript directly, with no authentication and no expiry.

Impact

Sensitive meeting audio/video and verbatim transcripts are accessible to anyone with the URL, indefinitely. Combined with F-01 (meeting ids are easy to obtain) this is a meaningful data-exposure path.

Recommended fix

Serve recordings/transcripts through an authenticated route that checks meeting ownership/membership and returns a short-lived pre-signed R2 URL, instead of a permanently-public bucket. If a public bucket must be retained short-term, use unguessable random object keys (not the meeting id) plus a lifecycle/expiry policy.

F-04 [MEDIUM] No rate limiting or request limits (Layer-7 / low-and-slow DoS)

Status Present

Location *src/app/api/{webhook,livekit-webhook,livekit-token}; src/inngest/function.ts:40; src/modules/meetings/server/procedures.ts:76*

What we found

No rate limiting, concurrency control or request-body size limit is applied to the public API routes. The webhook endpoints verify signatures (good) but can still be flooded; the token endpoint runs a Polar API call plus DB queries per request with no throttle. Server-side transcript fetches — fetch(transcriptUrl) in the Inngest job and getTranscript — have no timeout or response-size cap, so a slow/large response ties up the worker (a low-and-slow vector).

Impact

Resource exhaustion and increased cost (each meeting also dispatches an OpenAI Realtime agent). Primarily availability/billing impact rather than data compromise.

Recommended fix

Add IP/user rate limiting (e.g. Upstash Ratelimit or Vercel WAF) to all /api routes; cap request body size; add an AbortController timeout and a maximum-bytes guard to every server-side fetch of externally-stored transcripts.

F-05 [MEDIUM] Missing HTTP security headers

Status Present

Location *next.config.ts (empty config — no headers())*

What we found

next.config.ts defines no security response headers. There is no Content-Security-Policy, Strict-Transport-Security, X-Frame-Options/frame-ancestors, X-Content-Type-Options, Referrer-Policy or Permissions-Policy.

Impact

Reduced defence-in-depth: any future XSS is unconstrained by CSP, the app can be framed (clickjacking), and HSTS is not asserted. No single header is itself a breach, but their absence amplifies other issues.

Recommended fix

Add a headers() block (or middleware) setting at minimum Strict-Transport-Security, X-Content-Type-Options: nosniff, X-Frame-Options: DENY (or frame-ancestors 'none'), Referrer-Policy: strict-origin-when-cross-origin, a Permissions-Policy, and a tuned Content-Security-Policy.

F-06 [MEDIUM] Vulnerable dependencies

Status Present — npm audit: 67 findings (1 critical, 33 high, 32 moderate, 1 low)

Location *package-lock.json (transitive)*

What we found

A dependency audit reports 67 known vulnerabilities. High/critical advisories touch core packages including better-auth and drizzle-orm, plus common transitive libraries (axios, form-data, fast-uri, flatted, @grpc/grpc-js and the OpenTelemetry stack pulled in by the LiveKit agent). Some of these classes (e.g. axios/form-data) historically include ReDoS and request-smuggling issues — the most realistic ReDoS exposure for this project (see F-10).

Impact

Varies by advisory; because better-auth and drizzle-orm sit on the authentication and data paths, their advisories should be triaged first. Many OpenTelemetry/grpc advisories originate in the agent worker rather than the Next.js request path.

Recommended fix

Run `npm audit`, apply `npm audit fix` plus targeted version bumps (better-auth and drizzle-orm first), and add Dependabot/Renovate with a CI `npm audit --audit-level=high` gate. Prioritise web-facing packages over agent-only ones.

F-07 [LOW] Webhook replay — no nonce or idempotency

Status Partial — signatures are verified, but replay is possible

Location *src/app/api/livekit-webhook/route.ts (WebhookReceiver.receive); src/app/api/webhook/route.ts (streamVideo.verifyWebhook)*

What we found

Both webhook handlers correctly verify message signatures (LiveKit signs a JWT over a sha256 of the body; Stream uses an HMAC). Neither records a nonce/event-id or enforces a tight timestamp window, so a captured valid request can be replayed within the signature's validity window. The LiveKit join token is likewise reusable for its full 3600s TTL (src/lib/livekit.ts).

Impact

Low. Most handlers are effectively idempotent (status transitions are guarded by the current status; recordingUrl is overwritten with the same value). The most abusable case is replaying a Stream message.new event to re-trigger the AI chat reply.

Recommended fix

Persist processed webhook event ids and drop duplicates; reject events whose timestamp falls outside a small window (e.g. 5 minutes). Keep LiveKit token TTLs as short as the UX allows.

F-08 [LOW] Unescaped SQL LIKE wildcards in search

Status Present — not SQL injection, a LIKE-pattern issue

Location *src/modules/meetings/server/procedures.ts (getMany search); src/modules/agents/server/procedures.ts:102/117*

What we found

Search uses Drizzle's ilike(name, `%${search}%`). The value is safely parameterised (no SQL injection), but the user-supplied string is not escaped for LIKE metacharacters (% and _). A user can inject wildcards to broaden matches, and pathological patterns of many % make the scan more expensive.

Impact

Minor: altered search semantics and a weak DoS amplifier on large datasets. No confidentiality/integrity impact — results are already scoped to the caller's userId.

Recommended fix

Escape %, _ and the escape character in user input before building the LIKE pattern, and use an explicit ESCAPE clause.

F-09 [LOW] Polar billing client fixed to sandbox

Status Present — configuration

Location *src/lib/polar.ts:5 (server: "sandbox")*

What we found

The Polar SDK client is hard-coded to the sandbox server, so subscription/entitlement checks (premiumProcedure) run against sandbox data in every environment.

Impact

Not a direct breach, but premium gating cannot be trusted in production and sandbox tokens may behave unexpectedly. Flagged so it is not shipped to production unintentionally.

Recommended fix

Drive the Polar server mode from an environment variable and use the production server for production deployments.

F-10 [NOT PRESENT] SSTI & ReDoS (application code)

Status Not present in application code

Location *src/components/ui/chart.tsx (reviewed); src/modules/meetings/ui/component/completed-state.tsx:6*

What we found

SSTI: there is no server-side template engine (no Handlebars/EJS/Pug/Nunjucks). React/JSX auto-escapes, and the LLM-generated summary is rendered with react-markdown WITHOUT rehype-raw, so embedded HTML is not interpreted — there is no HTML/script injection sink. The single dangerouslySetInnerHTML (chart.tsx) is fed developer-defined chart config, not user input. ReDoS: no regular expression is built from user input; the only regex in the app is a static `.replace(/:/g, "")`. The realistic ReDoS exposure is therefore via vulnerable dependencies (F-06), not first-party code.

Recommended fix

No code change required. Keep react-markdown free of rehype-raw (or add a sanitiser if raw HTML is ever enabled) and address dependency advisories per F-06.

F-11 [NOT PRESENT] SQL / NoSQL injection

Status Not present

Location *All data access via Drizzle ORM (src/modules/**/server/procedures.ts, src/db/*)*

What we found

Every query uses Drizzle's parameterised query builder; there is no string-concatenated SQL and no sql.raw with user input. The few sql`` fragments (e.g. EXTRACT(EPOCH ...)) are static. tRPC inputs are validated with zod, and update mutations use tight zod allow-lists (meetingsUpdateSchema = {id,name,agentId}), preventing mass-assignment to sensitive columns. There is no NoSQL database in the stack. (See F-08 for the unrelated LIKE-wildcard nuance.)

Recommended fix

No change required. Continue routing all access through the ORM and validating inputs with zod.

F-12 [NOT PRESENT] Clipboard hijacking

Status Not present in application code

Location *Codebase-wide search for navigator.clipboard / execCommand / copy handlers*

What we found

No first-party code reads from or writes to the clipboard, and there are no copy/paste interception handlers. The third-party stream-chat-react widget may expose its own copy affordances, but those are library-managed and out of scope.

Recommended fix

No change required. If clipboard features are added later, never write secrets/tokens to the clipboard and treat pasted content as untrusted.

F-13 [INFO] Prompt injection into LLM summary & post-meeting chat

Status Informational — outside the requested classes, noted for completeness

Location *src/app/api/webhook/route.ts (message.new handler); src/inngest/function.ts (summariser prompt)*

What we found

Meeting transcripts, agent instructions and the generated summary are concatenated into LLM prompts via template literals. A participant could craft speech/text designed to manipulate the summariser or the post-meeting assistant (classic prompt injection). Because the output is rendered as escaped markdown (no rehype-raw), the blast radius is limited to misleading text, not XSS.

Recommended fix

Treat transcript/user text as untrusted data in prompts (clear delimiters, system-prompt hardening), keep rendering LLM output as sanitised markdown, and optionally add output validation.

---

4. Prioritised Remediation Roadmap

Immediate (this week)

- F-01 — Add a meeting ownership/membership check to /api/livekit-token before issuing a token. (Highest impact.)
- F-02 — Set and enforce BETTER_AUTH_SECRET in production; fail fast if missing.

Short term

- F-03 — Move recordings/transcripts behind an authenticated route with short-lived pre-signed URLs (or unguessable keys + expiry).
- F-05 — Add security headers in next.config.ts.
- F-06 — Triage and patch dependency advisories (better-auth, drizzle-orm first); add a CI audit gate.
- F-04 — Add rate limiting to /api routes and timeouts/size-caps to server-side fetches.

When convenient

- F-07 — Webhook idempotency (event-id dedupe + timestamp window).
- F-08 — Escape LIKE wildcards in search input.
- F-09 — Drive Polar server mode from an environment variable.
- F-13 — Harden LLM prompts against injection.
